

Министерство науки и высшего образования РФ Пензенский государственный
университет

Кафедра «Системы автоматизированного проектирования»

ОТЧЁТ

по лабораторной работе №4

по дисциплине «Логика и основы алгоритмизации в ИЗ»

на тему «Бинарное дерево поиска»

Выполнили ст. Гр 25ВВВ4:

Волосунин Е.

Колос М.

Приняли:

Юрова О.В.

Деем М.В.

Пенза 2026

Задание 1. Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.

Задание 2. Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.

Задание 3. *Изменить функцию добавления элементов для исключения добавления одинаковых символов.

1. Представленный код на Си был переписан на C++ с исправлением опечаток. Алгоритм поиска вводимого значения был реализован с помощью рекурсии, где на каждой ее итерации мы сравниваем число для поиска с числом в узле дерева и идем дальше в нужном направлении, если число нам не подходит.

2. Функция подсчета числа вхождений также была реализована с помощью рекурсии, где на каждой итерации мы проходим во все стороны и возвращаем единицу, если число найдено, и 0 если нет, при этом увеличивая на этот результат счетчик count.

3. В функцию добавления был добавлен флаг, обозначающий выбор соответствующего режима. Функционал был реализован элементарно - если в ходе добавления элемента мы встречаемся с тем же числом, что обязательно если оно существует, мы ничего не делаем и возвращаем указатель на существующий узел с этим числом.

Задание 4. *Оценить сложность процедуры поиска по значению в бинарном дереве.

В худшем случае, для поиска значения нам потребуется совершить $n = h$ операций, где h - глубина дерева. Например если все числа вводятся по возрастанию или убыванию, и мы ищем последний. Худший случай - $O(h)$.

В лучшем случае нам потребуется одна операция - искомый элемент является первым в дереве. Лучший случай - $O(1)$;

В среднем в сбалансированном дереве на каждом шагу мы отсекаем половину элементов, для подсчета сложности можно пойти с конца: на последнем шаге у нас 1 элемент, на предыдущем - 2, до этого - 4, дальше 8 и т.д. до n элементов. В сбалансированном дереве в среднем n элементов, и мы совершаем h шагов, $2^h = n$; $h = \log_2 n$. Средняя сложность - $O(\log n)$;

Псевдокод:

СТРУКТУРА Node:

data: целое число

left: указатель на Node (по умолчанию null)

right: указатель на Node (по умолчанию null)

// Вставка элемента в дерево поиска

ФУНКЦИЯ insert(node, value, noDuplicates):

ЕСЛИ node == null:

node = СОЗДАТЬ Node(value)

ВЕРНУТЬ node

ЕСЛИ noDuplicates И value == node.data:

ВЕРНУТЬ null // дубликат запрещен

ЕСЛИ value > node.data:

ВЕРНУТЬ insert(node.right, value, noDuplicates)

ИНАЧЕ:

ВЕРНУТЬ insert(node.left, value, noDuplicates)

// Вывод дерева на экран (обратный симметричный обход с отступами)

ФУНКЦИЯ printTree(node, level = 0):

ЕСЛИ node == null:

ВЕРНУТЬ

printTree(node.right, level + 1)

НАПЕЧАТАТЬ (level * 2 пробела) + node.data

printTree(node.left, level + 1)

// Поиск значения в BST

ФУНКЦИЯ search(node, value):

ЕСЛИ node == null ИЛИ node.data == value:

ВЕРНУТЬ node

ЕСЛИ value > node.data:

ВЕРНУТЬ search(node.right, value)

ИНАЧЕ:

ВЕРНУТЬ search(node.left, value)

// Подсчет количества вхождений значения

ФУНКЦИЯ countOccurrences(node, value):

ЕСЛИ node == null:

ВЕРНУТЬ 0

```
count = (node.data == value) ? 1 : 0
```

```
ВЕРНУТЬ count + countOccurrences(node.left, value) + countOccurrences(node.right, value)
```

```
// Освобождение памяти
```

```
ФУНКЦИЯ clearTree(node):
```

```
    ЕСЛИ node == null:
```

```
        ВЕРНУТЬ
```

```
    clearTree(node.right)
```

```
    clearTree(node.left)
```

```
    УДАЛИТЬ node
```

```
    node = null
```

```
// Генерация дерева случайных чисел
```

```
ФУНКЦИЯ generateTree(node, size, noDuplicates):
```

```
    size = МИНИМУМ(size, 100)
```

```
    ПОКА size > 0:
```

```
        val = СЛУЧАЙНОЕ_ЧИСЛО(1, 100)
```

```
        ЕСЛИ insert(node, val, noDuplicates) != null:
```

```
            size = size - 1
```

```
// Главный алгоритм
```

```
АЛГОРИТМ main:
```

```
    root = null
```

```
    ВВЕСТИ mode          // 0 - ручной ввод, 1 - генерация
```

```
    ВВЕСТИ noDuplicates  // 0 - разрешать дубликаты, 1 - без дубликатов
```

```
    ЕСЛИ mode == 0:
```

```
        ПОКА читается val:
```

```
            ЕСЛИ val == -1:
```

```
                ПРЕРВАТЬ
```

```
                insert(root, val, noDuplicates)
```

```
    ИНАЧЕ:
```

```
        ВВЕСТИ size
```

```
        generateTree(root, size, noDuplicates)
```

```
    НАПЕЧАТАТЬ "Дерево:"
```

```
    printTree(root)
```

ВВЕСТИ action // 0 - проверка наличия, 1 - подсчет вхождений
ВВЕСТИ target

ЕСЛИ action == 0:

 ЕСЛИ search(root, target) != null:

 НАПЕЧАТАТЬ "Число существует"

 ИНАЧЕ:

 НАПЕЧАТАТЬ "Число не существует"

ИНАЧЕ:

 НАПЕЧАТАТЬ "Вхождений: " + countOccurrences(root, target)

clearTree(root)